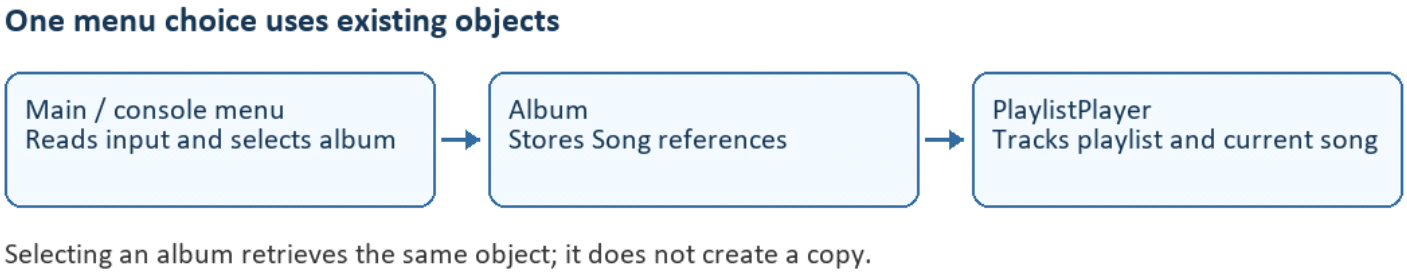
Lecture 6 - Dynamic Listenify and JSON APIs

Wednesday, October 14, 2026 3:58 PM

1. Dynamic Listenify: state and responsibilities

A dynamic console application reads choices and values from the user instead of creating only hardcoded sample objects. Albums and the player are created once before the menu loop so their state survives each choice. Data is stored in memory and is lost when the process ends.



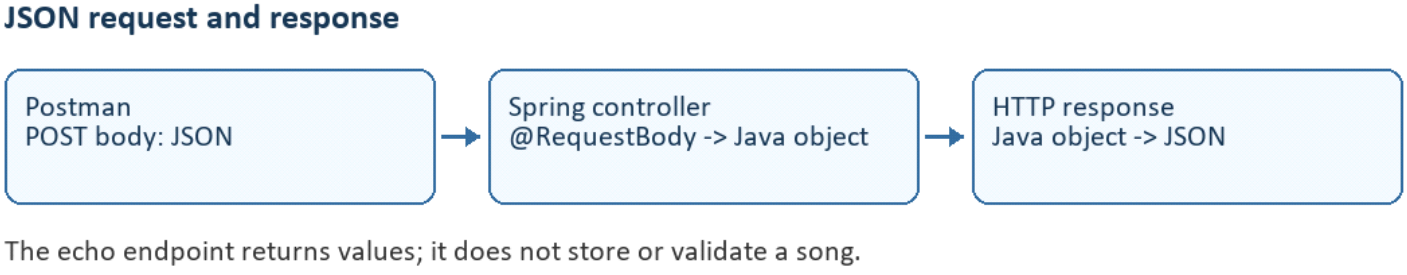
Main manages the album collection and Scanner input. Album manages one album and its songs. PlaylistPlayer manages playback position. Song validates its title and positive, finite duration.
Menu: 0 quit; 1 next; 2 previous; 3 replay; 4 playlist; 5 remove current; 6 pause/resume; 7 add song to an album, optionally to playlist; 8 create album; 9 show albums.
Album choices begin at 1; list indices begin at 0. Validate the choice before albums.get(albumNumber - 1). Inside case 7, if (albums.isEmpty()) break exits the switch and returns to the next menu iteration.
Q: Does adding a song with n put it in the playlist? No; it stays in the selected album only. Removing a playlist entry leaves the album song available. Pause/resume is a console simulation, not real audio playback.

2. ListIterator and the current song

The iterator cursor sits between elements. The player keeps it immediately after the current song. next() crosses the following song. To move to the previous song, cross current first, select its predecessor, then restore the cursor after the new current. At the first song, restore the cursor and report the boundary.
Song A | cursor | Song Bcurrent = Song Anext() returns Song B; cursor moves after Song B.
Insertion uses cursor.add(song), not a direct edit to the backing list while its iterator is active. The new entry goes immediately after current, so repeated additions without moving can reverse the order of the new entries. Removal uses cursor.remove().

3. A Java object becomes a JSON response

JSON is a text format for exchanging values. A JSON object uses { }; a JSON array uses [ ]. The client receives a representation of Java values, not a Java reference. In this web application, @RestController uses HTTP message conversion to write the returned object as JSON. toString() is not responsible for JSON conversion.



SongPreview.java — separate model class in the Spring project

package com.accio.lecture5;public class SongPreview { private String title; private double durationMinutes; public SongPreview() { } public SongPreview(String title, double durationMinutes) { this.title = title; this.durationMinutes = durationMinutes; } public String getTitle() { return title; } public void setTitle(String title) { this.title = title; } public double getDurationMinutes() { return durationMinutes; } public void setDurationMinutes(double durationMinutes) { this.durationMinutes = durationMinutes; }}
Getters expose response properties. For the simple mutable class used here, the no-argument constructor and setters support receiving input. The parameterized constructor creates sample response objects. Other Java binding designs are possible.
GET /preview-song -> 200, application/json{"title":"Sunrise","durationMinutes":3.5}
Q: Does returning an object save it? No. Does the browser run its Java constructor? No; that happens on the server. JSON property order may differ; compare the values.

4. A list becomes a JSON array

GET /preview-songs returns a List<SongPreview> as an array. Each request creates a fresh sample list. Refreshing does not append songs or store data. An empty list returns [] with 200; this is different from a missing endpoint.
[{"title":"Sunrise","durationMinutes":3.5}, {"title":"Moonlight","durationMinutes":4.0}]
A list response may contain zero or more songs. An empty successful result is 200 with [].

5. Product quote: query input and calculated output

GET /product-quote?name=Notebook&unitPrice=40&quantity=3 returns name, unitPrice, quantity and subtotal. subtotal = unitPrice × quantity.
A calculated getter can expose a JSON property without storing another field. This endpoint calculates a quote; it does not place an order.

6. Receiving JSON with @RequestBody

@RequestParam reads URL query values. @RequestBody reads and converts the HTTP body into a Java object. @PostMapping selects the POST route. Neither annotation stores data or automatically enforces our business rules.
POST http://127.0.0.1:8080/echo-song Content-Type: application/json{"title":"Road Trip","durationMinutes":4.2}Response: 200, same values as JSON.
Postman: POST -> enter the URL -> Body -> raw -> JSON -> enter the payload -> Send. Use JSON property names title and durationMinutes. The browser address bar normally makes a GET request, so use Postman for this JSON POST.
Q: Why 200 rather than 201? This endpoint echoes data and does not create a stored resource. Does POST change GET /preview-songs? No. A negative duration is currently echoed because SongPreview has no business validation.

7. Complete Spring controller

Lecture6PreviewController.java belongs under the existing Spring application package tree.

The console Listenify application and the Spring preview are separate applications.
package com.accio.lecture5;import java.util.ArrayList;import java.util.List;import org.springframework.web.bind.annotation.*;@RestControllerpublic class Lecture6PreviewController { @GetMapping("/preview-song") public SongPreview song() { return new SongPreview("Sunrise", 3.5); } @GetMapping("/preview-songs") public List<SongPreview> songs() { List<SongPreview> songs = new ArrayList<>(); songs.add(new SongPreview("Sunrise", 3.5)); songs.add(new SongPreview("Moonlight", 4.0)); return songs; } @GetMapping("/product-quote") public ProductQuote quote(@RequestParam("name") String name, @RequestParam("unitPrice") double unitPrice, @RequestParam("quantity") int quantity) { return new ProductQuote(name, unitPrice, quantity); } @GetMapping("/sample-quotes") public

List<ProductQuote> quotes() { List<ProductQuote> quotes = new ArrayList<>(); quotes.add(new ProductQuote("Notebook", 40, 3)); quotes.add(new ProductQuote("Pen", 10, 2)); return quotes; } @PostMapping("/echo-song") public SongPreview echo(@RequestBody SongPreview song) { return song; }}

8. Music Practice API — one combined exercise

PROBLEM: Build a small Music Practice API using TWO controller classes in your existing Spring Boot application. Reuse SongPreview from today. Keep the existing application main and preview/echo endpoints. This API lists sample songs and calculates a playlist summary; no database or saved playlist is required.
SongCatalogController: (1) GET /practice/sample-song returns one SongPreview; (2) GET /practice/songs returns three sample songs as a JSON array. An optional title query filters by exact title, ignoring case; no match returns an empty array. Without title, return all three.
PlaylistSummaryController: POST /practice/playlist-summary receives playlistName and a songs array in JSON. Return a NEW object containing playlistName, songCount and totalDurationMinutes. Use a loop to calculate the total. Empty songs returns count 0 and total 0.0.
Create PlaylistSummaryInput with a no-argument constructor, getters and setters, and PlaylistSummaryResponse with a constructor and getters. Reuse SongPreview for each song; do not modify the console Listenify Song. Assume a nonblank playlist name, a non-null songs array and valid song values. Validation and persistence are later lessons.
Query hint: @RequestParam(name = "title", defaultValue = "") supplies an empty string when the query is absent. If blank, return the full list; otherwise build a matching list using equalsIgnoreCase. Keep both controllers under the existing Boot application package tree, with distinct paths. Match the actual package: com.accio.springProject in the local project.

```
GET /practice/sample-song -> {"title":"Sunrise","durationMinutes":3.5}
GET /practice/songs -> three entries
GET /practice/songs?title=sunrise -> one-entry array
GET /practice/songs?title=Missing -> []

POST /practice/playlist-summary
Content-Type: application/json
{"playlistName":"Morning","songs":[{"title":"Sunrise","durationMinutes
":3.5},{"title":"Road Trip","durationMinutes":4.0}]}

200 response:
{"playlistName":"Morning","songCount":2,"totalDurationMinutes":7.5}
```

Also test {"playlistName":"Empty","songs":[]} -> count 0, total 0.0. The GET catalog stays unchanged after POST. Do not build a second Boot main, add dependencies, store songs or implement PATCH for this task.
Discussion: Can two controllers run in one application? Yes, Spring discovers both under the application package tree. Why different paths? The HTTP method and route must identify a handler unambiguously. Why is the filtered response still an array? Its Java return type is a list. Does POST save the playlist? No, this endpoint calculates a summary. Why two model classes? Input contains individual songs; output contains calculated fields.

Reference solution — compare after completing the exercise

PlaylistSummaryInput.java

```
package com.accio.springProject;
import java.util.List;

public class PlaylistSummaryInput {
    private String playlistName;
    private List<SongPreview> songs;

    public PlaylistSummaryInput() { }
    public String getPlaylistName() { return playlistName; }
```

```
    public void setPlaylistName(String playlistName) { this.playlistNa
me = playlistName; }
    public List<SongPreview> getSongs() { return songs; }
    public void setSongs(List<SongPreview> songs) { this.songs = songs
; }
}
```

PlaylistSummaryResponse.java

```
package com.accio.springProject;

public class PlaylistSummaryResponse {
    private final String playlistName;
    private final int songCount;
    private final double totalDurationMinutes;

    public PlaylistSummaryResponse(String playlistName, int songCount,
                                   double totalDurationMinutes) {
        this.playlistName = playlistName;
        this.songCount = songCount;
        this.totalDurationMinutes = totalDurationMinutes;
    }
    public String getPlaylistName() { return playlistName; }
    public int getSongCount() { return songCount; }
    public double getTotalDurationMinutes() { return totalDurationMinu
tes; }
}
```

SongCatalogController.java

```
package com.accio.springProject;
import java.util.ArrayList;
import java.util.List;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class SongCatalogController {
    @GetMapping("/practice/sample-song")
    public SongPreview sampleSong() {
        return new SongPreview("Sunrise", 3.5);
    }

    @GetMapping("/practice/songs")
    public List<SongPreview> songs(
            @RequestParam(name = "title", defaultValue = "") String ti
tle) {
        List<SongPreview> catalog = new ArrayList<>();
        catalog.add(new SongPreview("Sunrise", 3.5));
        catalog.add(new SongPreview("Road Trip", 4.0));
        catalog.add(new SongPreview("Moonlight", 2.5));

        if (title.isBlank()) {
            return catalog;
        }
        List<SongPreview> matches = new ArrayList<>();
        for (SongPreview song : catalog) {
            if (song.getTitle().equalsIgnoreCase(title)) {
                matches.add(song);
            }
        }
```

```
        return matches;
    }
}
```

PlaylistSummaryController.java

```
package com.accio.springProject;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class PlaylistSummaryController {
    @PostMapping("/practice/playlist-summary")
    public PlaylistSummaryResponse summarize(@RequestBody PlaylistSumm
aryInput input) {
        double total = 0.0;
        for (SongPreview song : input.getSongs()) {
            total += song.getDurationMinutes();
        }
        return new PlaylistSummaryResponse(input.getPlaylistName(),
                input.getSongs().size(), total);
    }
}
```

9. Practice and reading

Listenify: complete the dynamic homework and test empty player, first/last navigation, invalid menu input, unavailable album, duplicate song, album-only addition and playlist removal. Confirm failed validation does not add an album/song.
Assignment: complete the two-controller Music Practice API in point 8. Record the method, URL, request JSON where relevant, status and response body for each acceptance case. Commit the work on your own branch and submit through the usual class process.
Returning response bodies — Spring reference
Receiving request bodies — Spring reference